Target Specification, Not Model Capability, Explains Most Unit-Test Generation Failures at Medium Complexity

Nguyễn Tùng Ân, Đoàn Thị Thu Kim, Sơn Hải, Trương Hoàng Phúc, and Trần Xuân Lộc

Group 2 — SE1944, FPT University, Vietnam
Supervisor: L.T.Q.Chi

Abstract. Failures of LLM-generated unit tests are usually recorded against the model. Examining them individually, we find most belong to the benchmark and the measurements instead. Sampling by cyclomatic complexity alone admits targets no external test can reach: of 1,848 functions in the band $5 \leq CC \leq 10$ across ten projects, only 474 — 26 % — are simultaneously public and unambiguously named. We rebuild the benchmark under five testability criteria fixed in advance, sample 60 Java and 60 Python functions ($n = 59$ for Java after excluding one mis-sampled target), and score every suite at four nested tiers, resting all claims on mutation score inside the target's line range — the only tier a suite that verifies nothing cannot satisfy. Two one-shot prompts differing in exactly one variable isolate the effect of supplying the target specification. On Python it raises the proportion of suites reaching the target from 29/60 to 42/60 ($p = 0.003$, rank-biserial +0.61), but the proportion detecting a fault only from 16/60 to 22/60 ($p = 0.33$): the prompt improves arrival, not discrimination — a split branch coverage alone would have concealed. Against established generators the ordering reverses by language: the model beats Pynguin decisively on Python, where Pynguin detected faults on 2 of 60 targets under either configuration ($p \leq 0.003$ in every pairing), while on Java both EvoSuite and Randoop detect faults on more targets, significantly so only for EvoSuite under the stricter gate ($p = 0.028$). Six measurement defects surfaced, four introduced while repairing the other two; none produced an error message.

Keywords: Unit test generation · Large language models · Mutation testing · Cyclomatic complexity · Measurement validity

1 Introduction

A large language model asked to write a unit test for a particular function will often produce something that does not compile. It is tempting to blame the model, and we read it the same way in an earlier analysis of `gpt-4o-mini` on 120 medium-complexity functions. This paper reports on the investigation that

followed, and explains why the answer is mostly about a benchmarking artifact rather than the model itself.

We found two issues, and together they compound.

Benchmarks contain targets that no generator can test. Sampling functions by cyclomatic complexity selects for difficulty, but says nothing about whether a function is reachable from an external test. If it is not, no generator can hope to test it, and asking one to do so simply labels it as failed. We found that in the ten projects studied, 1,848 functions fell in the desired complexity range of $5 \leq CC \leq 10$. Only 474 of them, about a quarter, were simultaneously public and unambiguously named — so a generator that failed on the remaining 74 % would have been asked to do what no tool is capable of doing. Likewise, attributing a method to the wrong declaring class renders a test suite invalid, and there is ample opportunity for that to happen: a brace counter that does not decrement on block exit will assign the method to whichever nested type was last opened.

The usual metrics can be satisfied with no testing at all. Compilation success and coverage are both metrics that can be reported by a suite that never tests anything. Reflection turns every identifier into a string, so the compiler stops checking it, and a call naming a method that does not exist compiles cleanly with a zero return code and fails only at run time. Separately, a generated suite in our own corpus makes a single assertion-free call to the wrong function in the same module: it compiles, it collects, and it passes, so a green-test criterion records it as a success. The issue is not that these metrics are noisy — it is that a number can still be reported after the underlying measurement has stopped working, with nothing to signal it. We found six such cases, and in four of them the defect was ours, introduced while repairing the other two (Section 6).

This paper

We build a new benchmark satisfying five testability criteria established in advance, sample 60 Java and 60 Python functions from the resulting set, and evaluate each test suite at four nested levels. We rest every claim on mutation score within the target's line range, the only level that a suite testing nothing cannot satisfy. Two variants of the prompt differ in exactly one variable, the information given about the target, so that the contribution of the target specification is attributable to it; three established tools (EvoSuite, Randoop, Pynguin) act as reference points. The details of sampling, tool versions, budgets, and hypotheses were fixed in a public repository before measurements began.

Three findings stand out.

1. **Specification improves reach, not discrimination.** Adding a fully qualified target name, its signature, and a working constructor call to the prompt increases the proportion of suites reaching the target in Python from 29/60 to 42/60 ($p = 0.003$, rank-biserial +0.61). The number detecting a fault rises from 16/60 to 22/60, but this is not significant ($p = 0.33$). The prompt gets

the test to the right place; it does not make it check anything. A study that used coverage as its sole metric would have announced the change as an unambiguous improvement.

2. **LLMs and search-based tools win in different languages.** In Python, both prompt conditions beat Pynguin across the board: Pynguin detects a fault on 2/60 targets against the model's 16/60 and 22/60, better in every pairing at $p \leq 0.003$, and this holds under both of the configurations we tried for Pynguin. The ordering reverses in Java, where both search-based tools detect faults on more targets than the model — though the paired test reaches significance only for EvoSuite, and only under the stricter gate. These findings do not support one family over the other in general.
3. **Most of the attrition is structural.** The binding constraint was never writing assertions. It was obtaining an instance of the subject under test in a state where the behaviour of interest is observable — the maze of object creation that the search-based literature has acknowledged for years. That is the one barrier the model was unable to get past, and supplying it a verified constructor was not enough.

We report the full sample size at all stages of the benchmarking funnel, and we report both configurations of Pynguin rather than the better one. We also describe a defect in the Java implementation of the green-test gate — the criterion requiring at least one test to pass against the unmodified source — which we found to contradict the definition we had registered in advance, and which we corrected. Where that repair favours us, we say so and report the outcome under both settings, so that nothing in this paper depends on the reader's preference between the two.

2 Related Work

2.1 Search-Based and Random Test Generation

Automated unit-test generation has a long prehistory preceding the advent of language models. Randoop creates test sequences incrementally and uses feedback from executing each prefix to discard sequences that are illegal or redundant [11]. EvoSuite treats an entire suite as an individual in a genetic algorithm and optimizes it using a coverage-based fitness function [3]. Both approaches have been intensively evaluated at scale: EvoSuite demonstrated its ability to achieve high branch coverage across a large sample of open-source classes, while also revealing that a significant proportion of classes remain hard for reasons unrelated to the search itself [4]. Pynguin adapts the search-based approach to Python, where the lack of static typing decreases the information available to the test generator [8,9].

Two findings from this line of work are directly pertinent to our own. First, coverage and fault detection are not interchangeable: Shamshiri et al. find that automatically generated suites with high coverage detect only a minority of real faults [14], and Inozemtseva and Holmes find that coverage's correlation with

suite effectiveness is significantly lower than often assumed [5]. Secondly, the difficulty of a target is not captured by its complexity: constructing the receiver object often proves to be the major stumbling block for test generation. We return to both observations in Section 3.3 and Section 5 respectively.

2.2 LLM-Based Test Generation

Code-trained language models transformed test generation into a prompting task [2]. The earliest approaches in this space involved fine-tuning a transformer on method–test pairs with surrounding focal context [16]. Later works switched to prompting. TestPilot produces JavaScript tests from documentation and usage examples and employs a requery mechanism triggered by test failures [13]. ChatTester introduces an iterative repair loop that capitalizes on compiler and runtime errors, yielding markedly better results than a single-shot approach [18]. Siddiq et al. evaluate several LLMs' ability to generate JUnit tests and find compilation failures to dominate over weak assertions as a source of failures [15]. Finally, a recent survey enumerates the space and reflects on evaluation practices, noting how unevenly they are applied [17].

Our study is narrower than most of these works in one respect (a single API call per function) and stricter in another (no iterative repair and no retrieval). We exclude the repair loop deliberately, because it conflates the quality of the initial generation with the quality of the error signal fed back into it — and when compilation failure is itself the object of study, that conflation is fatal. Where these systems ask how high the success rate can be driven, we ask what the initial generation is actually missing, and whether it is something a prompt can supply. The two prompt conditions of Section 3.2 therefore differ in exactly one variable, so that whatever difference appears is attributable to that variable.

2.3 Evaluation Metrics and Their Validity

Mutation analysis is the canonical means of assessing test-suite adequacy in fault-based evaluation. Mutants have been shown to be a reasonable if imperfect substitute for real faults in controlled experiments [6,12]. It is also computationally demanding, which is why many works on LLM-based test generation focus on compilation success and coverage as surrogates for adequacy.

This substitution forms the theoretical basis for our own contribution. Compilation success and coverage are easy to measure, but they ask less than one might hope, and what they ask can be satisfied without testing anything: a reflective call compiles regardless of whether the named method exists, and a suite that constructs no object and calls nothing still passes. Neither case actually evaluates the code under test, yet both are recorded as successes by the metrics used in existing work. We observed both in our own corpus (Section 3.3), which is why we report four nested tiers and rest every claim on the only tier that cannot be satisfied vacuously.

Another subtlety involves the validity of the benchmark under consideration. Selecting targets based on cyclomatic complexity [10] is common practice,

but complexity is not indicative of reachability. A method can be private, can have multiple overloads sharing a name, or can belong to a class that cannot be instantiated from outside. A suite that fails on such a target tells us about the sampling, not about the generator. To our knowledge, the proportion of complexity-sampled methods addressable from an external test is rarely quantified; we report the funnel for that reason, and its attrition turns out to be the study's most transferable number.

2.4 Methodological Practice

Because the current work re-examines a dataset after having seen results obtained on an earlier one, the distinction between repairing an instrument and selecting a favourable outcome is particularly crucial. Kerr's articulation of the hazards of hypothesising after the results are known [7] describes the failure mode precisely. We address it in two ways. We committed our sampling criteria, tool versions, budgets, and hypotheses to the repository before any measurement was run. And the earlier study's dataset and reported numbers are left unchanged: this paper stands beside it rather than replacing it. For the statistical evaluations we adhere to the recommendations of Arcuri and Briand regarding non-parametric tests and effect sizes for randomised algorithms in software engineering [1].

3 Methodology

3.1 Dataset Construction

We extracted functions from ten open-source repositories at a particular commit (eight Java projects: `commons-cli`, `commons-csv`, `commons-collections`, `commons-math`, `gson`, `jsoup`, `joda-time`, `jfreechart`; two Python projects: `flask`, `requests`), using Lizard 1.23.0 to compute cyclomatic complexity (CC). We select the medium complexity band $5 \leq CC \leq 10$, as in our previous work.

While complexity is a precondition for being a challenging test target, a study sampling on complexity confounds two phenomena: the quality of tests produced by the generator, and whether the sampled target is reachable by the generator at all. We therefore applied five additional criteria, each fixed before any test was generated, and each considered necessary for an LLM, a search-based generator, or a human to reach the target through the public API.

F1 — **Non-trivial body.** $nloc \geq 3$. This filters out stubs, abstract methods, and interface signatures that specify no branches.

F2 — **Public or exported.** Java: declared `public`. Python: not prefixed with an underscore. A `private` method cannot be invoked directly by an external test class; reflection can reach it, but then the compiler no longer checks any name in the test (Section 3.3), so compilation ceases to be evidence of anything.

Table 1. Sampling funnel. Each row reports how many candidates survive the criterion. F4 corrects the declaring-class attribution that F3 depends on rather than removing candidates in its own right, so it has no separate row.

Stage	Criterion	Remaining
F0	$5 \leq CC \leq 10$ (Lizard)	1,848
F1	non-trivial body	1,848
F3	unambiguous name	753
F2	public / exported	474
F5	dynamically resolvable (Python)	64
	final pool	Java 406, Python 64
	sampled	Java 60, Python 60

F3 — Unambiguous name. Java: the method name occurs exactly once in its declaring class. Python: the function name occurs exactly once in its module. An overloaded name cannot appear in a prompt without additional disambiguating context, and the compiler rejects the resulting call as ambiguous.

F4 — Verified declaring class. The enclosing class is found with a brace-depth stack that *pops* on block exit. Taking "the nearest enclosing class declaration" without popping would attribute a method to whichever nested class was most recently declared in the file.

F5 — Dynamically resolvable (Python). The module is imported, the qualified name is walked with `getattr`, and the result is confirmed callable. Syntactic resolution is not sufficient evidence for run-time binding.

Table 1 shows the complete funnel. We report every count-reducing stage rather than only the final count, because the attrition is informative: of the 1,848 functions that passed the complexity filter, only 474 (26 %) were both public and unambiguously named. Around three quarters of medium-complexity functions in these projects are not valid test targets for an external generator.

From the final pool we sample 60 Java and 60 Python functions, stratified across repositories (Java: 3–9 functions per project; Python: 32 from `requests`, 28 from `flask`). Complexity is skewed towards the lower end of the band (CC=5: 56 functions), and the median function contains 17 lines of code. Of the Python targets, 34 are instance methods and 26 are module-level functions. Every one of the 120 functions was checked for run-time accessibility before generating tests. One Java target was later found to violate F2 — our own parser had recorded a `protected` method as public — and is excluded, so Java results are reported over $n = 59$ and Python over $n = 60$ (Section 6).

3.2 Test Generation

All tests are generated by `gpt-4o-mini-2024-07-18` at `temperature` = 0, `top_p` = 1.0, `max_tokens` = 2048, in a **single call per function** with no feedback loop and

no retrieval. We compare two *one-shot* prompt conditions. Both contain exactly one worked exemplar (a trivial `add` function with its test), so both are one-shot in the usual sense, and both issue one call. They differ in exactly one respect:

V1 — baseline. The prompt contains the target function name and source. This is the prompt used in our earlier study.
V2 — target-specified. Identical to V1, with one block inserted before the task description: the fully qualified target name, its signature, and declaring class, plus — for instance methods — a constructor call that has been *executed* and verified to produce a valid receiver object.

Holding the worked exemplar fixed is crucial: an earlier variant of the enriched prompt accidentally omitted the exemplar and thus differed from V1 in two respects at once (it dropped the exemplar and it added the metadata), so no causal attribution was possible from it. We report that variant only as a secondary observation and label it zero-shot, since it is one.

3.3 Measurement: Four Tiers

A single "valid / invalid" flag conflates outcomes that differ in kind, so we report four nested tiers. Each subsequent tier implies the one before it, and — importantly — the first two can be satisfied by a test suite that does not test the target at all.

T1 — Compiles / collects. The test file is syntactically and type-correct.
T2 — At least one green test. At least one test passes against the unmodified source.
T3 — Reaches the target. Branch coverage > 0 within the target's line range $[start, end]$.
T4 — Detects faults. Mutation score > 0 within the same line range.

Only T4 speaks to a suite's ability to test the target; we demonstrate that T1 and T2 can be satisfied by suites that verify nothing:

- **T1 under reflection.** Reflection turns every name into a string, so the compiler stops checking it. A call to `getDeclaredMethod("aNameThatDoesNotExist")` compiles with return code 0 and fails only at run time. Any compilation-based validity rate is therefore uninformative for suites that use reflection.
- **T2 under an assertion-free suite.** The search-based suite generated for `requests.hooks` (target `dispatch_hook`) contains exactly one test, and that test is a single bare call to a *different* function in the same module, with no assertion of any kind. It compiles, it collects, and it passes — T1 and T2 are both satisfied — while measured branch coverage inside the target's line range is 0. A green-test criterion records this as a success.

The two tiers are therefore not merely weaker than T4; they can be satisfied by suites that are, respectively, unchecked by the compiler and empty of assertions.

Branch coverage is measured with `coverage.py` (Python), attributed to the target's line range rather than to the whole file; T3 is therefore reported for Python only. The Java pipeline executes the generated JUnit suites through the JUnit Platform launcher and rests on T4, so no Java branch-coverage figure is claimed anywhere in this paper. Mutation analysis applies the same operator set to both languages — arithmetic ($+/-$, $\times/\div$), relational ($>$ / $\geq$, $<$ / $\leq$), equality ($==/\neq$), boolean ($\wedge/\vee$), boolean constant inversion, and numeric constant increment — restricted to the target's line range and capped per function by language (20 mutants for every Java suite, 30 for every Python suite). The cap binds two Java functions and no Python one (Section 6); it does not affect T4, which asks only whether any mutant is killed.

Two procedural safeguards are worth stating because their absence silently corrupts results. First, mutation requires overwriting the source; we mutate a *copy* of the source tree placed ahead of the original on the import path, so that concurrent measurements reading the same tree are unaffected. Second, tool versions must be matched across families: JUnit 5 pairs Jupiter 5.x with Platform 1.x, and mixing a Jupiter 6 artifact with JUnit 5-style tests makes the launcher discover zero tests while still exiting successfully — a failure that reports as a legitimate zero.

3.4 Baselines

Each baseline generates per class (Java) or per module (Python), matching how the tools operate; results are then attributed to individual functions by line range, exactly as for the LLM suites.

EvoSuite 1.2.0 , 60 s per class, run under JDK 11. EvoSuite performs deep reflection into JDK internals, which the module system blocks from Java 9 onward, and its master process launches its client from `JAVA_HOME` — so both must point at the same JDK, or the client dies while the master still exits successfully. Because the projects were compiled with JDK 17, we recompiled sources to Java 11 bytecode into a separate output directory, leaving the original build untouched.

Randoop 4.3.3 , 60 s per class, JDK 17.

Pynguin 0.45.0 , 90 s per module, under **two configurations reported side by side**. In its default configuration Pynguin serialises module state to a worker process with `dill` and aborts on C-extension type objects that cannot be resolved by name, producing no output at all for several modules. We therefore also run it with the master–worker split disabled. Reporting both separates "the tool scored low" from "the tool never ran".

3.5 Statistical Analysis

Conditions are compared with the paired Wilcoxon signed-rank test, matching on function identifier, at $\alpha = 0.05$, with matched-pairs rank-biserial correlation as effect size. Java and Python are analysed separately throughout: the toolchains

Table 2. Fault-detection rate (T4), original versus rebuilt dataset, same prompt (V1).

Language	Green gate	Original	Rebuilt
Python	per-test	13/60 (21.7 %)	16/60 (26.7 %)
Java	whole-suite	6/60 (10.0 %)	12/59 (20.3 %)

differ, and Java enforces access control while Python does not, so their failure modes are not commensurable. Every rate is reported over the functions of its own language ($n = 59$ for Java after the exclusion above, $n = 60$ for Python).

Results are given at two levels in parallel: *all* ($n = 59$ for Java and $n = 60$ for Python, with unmeasurable cases scored 0) and *effective* (the subset with a non-zero value). The *all* level is reported even where the *effective* level is more favourable.

All configuration decisions above — criteria, tool versions, budgets, the two-branch Pynguin design, and the hypotheses — were recorded in a pre-registration committed before any baseline was executed.

4 Results

We report results per language across all metrics. Within a language, all rates are over the same n: 60 for Python, 59 for Java (after dropping the mis-sampled target in Section 6). The primary tier is T4, fault detection within the target's line range; we report T3, reaching the target, alongside it because the two diverge.

4.1 RQ-A: Clean Versus Original Dataset

Table 2 compares the baseline prompt's fault-detection rate on the original set with the same prompt on the rebuilt set. This is not a fair comparison — the two sets contain different target functions, and the rebuilt one is easier by construction — so we report only the magnitude of the gap, and only between measurements taken under the *same* green-test gate, since a gate mismatch on its own would inflate the difference.

The rebuilt set roughly doubles Java's rate and lifts Python's by about a quarter (13/60 to 16/60). Because the two samples are not matched, we draw no causal conclusion; the point is only that much of the original set's poor performance came from targets that no external test could reasonably cover, and removing them changes the headline number substantially.

4.2 RQ-B: Effect of Target Specification

Both prompt variations are one-shot; the difference is one block of target metadata (Section 3.2). Table 3 reports both tiers for Python, where the effect is strongest and splits most cleanly between them. We also report the median over the non-zero subset, since the non-zero results sit far above the zero group.

Table 3. Python, V1 versus V2. Paired Wilcoxon signed-rank, $\alpha = 0.05$; r_{rb} is matched-pairs rank-biserial.

Tier	V1	V2	p	r_{rb}	
T3 (reaches target)	29/60	42/60	0.0027	+0.61	significant
T4 (detects fault)	16/60	22/60	0.3259	+0.28	n.s.

The target specification increases the proportion of suites that reach the target (29/60 to 42/60, $p = 0.003$), but its effect on the proportion detecting the injected fault is not significant (16/60 to 22/60, $p = 0.33$). On the non-zero subsets, the medians of T3 and T4 barely change (75.0 to 79.2 and 84.4 to 88.9), which suggests the effect lies almost entirely in *how many* suites clear each tier rather than in how well the clearing suites perform. A coverage-only study would have read this as a clear improvement; the fault-based tier shows the improvement stops short of testing behaviour.

The pre-registered hypothesis H-B predicted that the enriched prompt would not outperform the baseline. It holds for T4 and is contradicted for T3. We report this as a partially refuted hypothesis, since the significant T3 effect runs in the direction H-B predicted against.

On Java, the same intervention has little effect. Under the per-test gate, both prompts detect the fault on 16 of 59 targets; under the whole-suite gate the counts are 12 (V1) and 10 (V2). The number of compile-failure targets is the same under both prompts, 38, and 35 of those are the same functions. The target block, which identifies the target uniquely, resolves the addressing problem but not the type-level issues that dominate Java compilation failures. Only 4 functions differ in a way the paired test could use, below the six-pair minimum for a Wilcoxon test; we report the observed counts and the mechanism (Section 5.1) rather than a p-value.

4.3 RQ-C: Against Established Generators

Table 4 pits the model against the search-based and random generators. Java is reported under both green-test gates; Python under both Pynguin settings. No analysis below depends on adopting one setting over the other.

The two languages disagree, and the disagreement is the result. On Python the model overwhelmingly outperforms Pynguin: against either Pynguin configuration, and using either prompt, the paired test rejects at $p \leq 0.003$, with rank-biserial correlations between -0.81 and -0.96. The reason is visible in the failure counts rather than the scores: Pynguin produced no suite at all for 39 of 60 targets in its default configuration, and disabling the master–worker split — the documented workaround for the serialisation crash responsible — reduced that to 35 while leaving T4 unchanged at 2/60. That net movement of four conceals considerable churn: 17 targets gained a suite and 13 lost one, and 14 of the 17 newly generated suites failed at collection.

Table 4. Fault detection (T4) of the model versus baselines. Java $n = 59$, Python $n = 60$. GPT columns are the target-specified prompt (V2); the paired test compares that column with each baseline.

Language Setting	GPT (V2)	Baseline Paired test (p, r_{rb})
Java — EvoSuite		
per-test	16/59	23/59 0.14, +0.32 (n.s.)
whole-suite	10/59	24/59 0.028, +0.51 (sig.)
Java — Randoop		
per-test	16/59	19/59 0.86, −0.04 (n.s.)
whole-suite	10/59	18/59 0.49, +0.18 (n.s.)
Python — Pynguin		
default	22/60	2/60 0.0001, −0.96 (sig.)
no split	22/60	2/60 0.0001, −0.96 (sig.)

On Java the count ordering reverses. Both baselines detect faults on more targets than the model in every pairing (EvoSuite 23 or 24 against the model's 16 or 10; Randoop 19 or 18 against 16 or 10). The paired Wilcoxon test, however, reaches significance in only one of the four comparisons: EvoSuite under the whole-suite gate ($p = 0.028$, $r_{rb} = +0.51$). The other three, including both Randoop comparisons, do not reject at $\alpha = 0.05$, with p ranging from 0.14 to 0.86. The Java evidence is therefore a consistent count advantage for the search-based tools that is statistically firm only for EvoSuite once the stricter gate is applied; it is not the decisive reversal the raw counts might suggest. Even so, it points the opposite way from Python, where the model rejects against Pynguin at $p \leq 0.003$ in every pairing. Java is the language these tools were designed for, and where static types give a search-based generator the call information the model was shown to lack (Section 5.1). We therefore report no single verdict on whether the model beats automated generators: the count ordering is opposite in the two languages, and a study limited to either one would have reached the other's headline.

5 Discussion

5.1 Three Barriers, Only the First of Which a Prompt Can Remove

The two prompt conditions differ by only one variable, and the effect of that variable cleanly splits between languages. For Python, adding the target specification cut the number of suites that failed to import or collect from 17 to 4 — 14 targets rescued and 1 newly broken. For Java, it made virtually no difference: 38 failed to compile with the baseline prompt, and 38 with the target-specified one, with 35 functions overlapping between those sets. Three functions moved in either direction, which is noise, not an effect.

The difference in effect between languages is not a peculiarity of their tools — it is encoded in the block itself. The block specifies the identity of the target:

its qualified name, declaring class or module, signature, and a constructor call for the receiver, if any. For Python, that is enough to remove barriers to invocation — at the stage these suites failed, the failure was always importing a module from an incorrect path, calling a non-existent function, or attempting to invoke a method on a class that does not support it. Give Python the correct name and location, and it will resolve the symbol successfully. Java performs these checks at compile time, so a correct name is still insufficient — the model must synthesise type-correct uses of the symbol within the suite, including receiver arguments of the correct arity and parameter types, return types, thrown exceptions, generics, and so on. The target block does not include any of that information.

This suggests a three-level reading of the results in terms of what the generated test must do to observe the target. We offer this as a hypothesis for future work:

1. **Addressing** — calling the right thing at all. The target specification removed this barrier, almost entirely in Python.
2. **Typing** — calling it in a way the compiler accepts. Only relevant to Java. The target specification does not meaningfully help, because the difficulty is not with the target itself, but the surrounding API.
3. **State** — constructing a receiver in a configuration where the behaviour of interest is visible. Neither prompt condition addresses this.

The reason to take this seriously is the evidence in support of the third item. For Python, the number of suites that imported successfully but had no passing test barely moved between the two prompt conditions: 13 under the baseline prompt, 14 under the target-specified one, with only 7 functions appearing in both sets. Removing the addressing barrier did not shrink the state barrier; it *exposed* it. The population changed while the size did not.

The observation that the barrier is not with assertion writing but with receiver construction is not new — it is the reason search-based test generation struggles with object-oriented code [4,14]. The point of this finding is that the same wall exists for language models, and that providing the constructor is not enough to get past it — our snippets were run and checked to create the receiver, and the state barrier persisted. Building an instance is not the same as building an instance in a state where some behaviour is triggered.

5.2 Reaching a Target and Testing It Are Different Achievements

The clearest finding of this study is the difference between two tiers that would have appeared identical in a single-metric study. On Python, the target specification increased the proportion of suites that reached the target from 29/60 to 42/60 ($p = 0.003$, $r_{rb} = +0.61$), but its effect on the number that could detect an injected fault was not significant, 16/60 to 22/60 ($p = 0.33$).

A study that used only coverage as a metric would have concluded that the enriched prompt is an improvement across the board, while also being wrong about what aspect of testing it improved. The suites arrived at the right function

more often, but tested it no more often. Coverage measures arrival and mutation score measures discrimination — only the latter is a metric of testing ability in any meaningful sense, and only the former was improved by the prompt change. That coverage is misleading about test-suite quality is a well-known finding [5]; what this experiment adds is a demonstration that a specific, plausible change to the prompt moves the two metrics by different amounts, and in a way that inverts the conclusion drawn from them.

This is also where the pre-registered hypothesis H-B begins to fall apart. We predicted, with no knowledge of the results, that the enriched prompt would be no better than the baseline. That prediction holds at T4 and fails at T3. We report this as a partially refuted hypothesis, rather than an overturned one, because the direction of effect at T3 is the one we predicted against.

5.3 Neither Family Wins Outright

The comparison with existing generators also fails to reveal a clear winner, and the languages rank oppositely.

On Python, both prompt conditions substantially outperformed Pynguin: 16/60 and 22/60 suites detecting a fault compared to 2/60, $p \leq 0.003$ in all pairs, with rank-biserial correlations ranging from -0.81 to -0.96. The reason for the gap was apparent in the numbers of failed suites: Pynguin failed to generate any suite for 39/60 targets in its default configuration. The documented workaround for the reported serialisation crash (disabling the master–worker split) reduced this to 35, a net movement of four that conceals considerable churn: 17 targets gained a suite and 13 lost one, and 14 of the 17 new suites failed at collection, leaving T4 at 2/60. We report this result because it is the sort of finding that disappears when a study only reports the better-performing configuration. In this case, neither configuration worked.

On Java, the count ordering was reversed: the search-based tools found faults on more targets than the model in all pairs (Section 4.3). The reversal was firmer for EvoSuite, for which the paired test reached significance at the whole-suite gate ($p = 0.028$), than for Randoop, for which no gate did; we interpret this as a consistent count advantage for the mature tools, rather than a uniformly significant one. Java is also where these tools were developed and where they have been iterated upon for the last decade, and where static typing gives them the information needed to construct calls — the information the language model was demonstrated to lack in Section 5.1. Python removes that information, and the ordering flips.

The conclusion is that neither family of tools is demonstrably better overall, but that they fail in ways that are specific to languages. A study that examined only Java would have confidently reported the superiority of search-based testing; a study that examined only Python would have confidently reported the opposite.

5.4 What This Means for Evaluating Test Generators

Three practices follow from what went wrong rather than from what worked.

Report the sampling funnel. Only 26 % of medium-complexity functions in these ten projects are both public and unambiguously named. A benchmark that samples on complexity alone will therefore spend most of its mass on targets that no external generator can address, and every tool evaluated on it is charged for failures that belong to the sampling rather than to the tool. The funnel costs nothing to report, and it bounds how much of a measured failure rate can be attributed to the tool at all.

Rest conclusions on a tier that cannot be satisfied vacuously. Both compilation success and the green-test criterion were defeated in this corpus. Compilation is defeated by reflection, which turns names into unchecked strings. The green-test criterion is defeated by a suite that makes a single assertion-free call to the wrong function: it compiles, it collects, and it passes, so the criterion records it as a success while branch coverage inside the target's line range is zero. Neither case requires an adversary; both appeared in ordinary tool output. Metrics should be interpreted with extra caution in the presence of reflection, or similar techniques that let code bypass static checks.

Assume the harness is broken until a failure proves otherwise. The failure mode that consumed the most time in this study was not a tool underperforming, but a metric misreporting. Hardcoded constants, validity flags erroneously indicating success, reflection removing the compiler's ability to check names, a version mismatch reporting no tests while exiting successfully, a JDK mismatch doing the same, and a green-test gate implemented differently in the two languages under one name appeared six times in total, four of which were introduced by us while attempting to fix the other two. None of these issues caused an exception to be thrown, and consequently, none of them were caught by the initial harness design. The requirement for a harness is that it must have a test that fails — if the pipeline below the metric is entirely broken but the metric reports no change, that metric is no longer useful. We enforce this requirement for all reported rates, and it is the cheapest such check we know of.

6 Threats to Validity

6.1 Construct Validity

What a validity flag measures. The main construct-validity concern is that our suite's "valid suite" marker does not capture what it ought to, and that we fail to detect when it does not. There were six cases, four of which were in our own instrumentation.

1. A compilation flag inferred from a coverage report only captures whether the class appears in it; coverage tools list all classes (including unreachable ones), which yield zero percent branch coverage, parsed as a successfully compiled function with zero coverage.
2. A criterion of "at least one test executed" erroneously counts a file in which all tests failed.
3. Reflection renders compiler-based name checking useless for any suite that uses it, so a compilation-based rate is uninformative there.
4. A discrepancy in tool versions can cause a test launcher to report zero tests found while exiting successfully.

The fifth case has ramifications for the statistical test, so we describe it in detail. Our Java runner and the tests it generated were compiled with the default JDK (class file version 61) and run on the JDK 11 required by EvoSuite (which reads class files up to version 55). The JVM therefore threw an `UnsupportedClassVersionError`. Because it was thrown inside a child process, the harness saw only the absence of a result line and reported "no tests ran" for every target. EvoSuite scored 0/60, and the paired test against it returned $p = 0.005$ with a rank-biserial correlation of -1.000; had we reported it, the paper would have erroneously claimed a significant advantage for the model over EvoSuite. The correct value is given in Section 4: a well-formed p-value made the artefact appear stronger than it was.

The sixth case is the green-test gate. Our Python implementation correctly kept the passing tests and discarded the failing ones, as T2 specifies; the Java version rejected the whole function if any test in it had failed. The two languages implemented different constructs under the same label, and the Java version contradicted our pre-registered definition. We fixed it and report both settings henceforth (Section 4); the correction is relevant to conclusion validity (see below), as it improved our own results.

None of these issues produces a visibly incorrect result; each merely returns a plausible number. We therefore report the four nested tiers of Section 3.3 instead of a single flag, and treat only T4 — fault detection within the target's line range — as evidence that a suite tests its target.

Mutation operators. Our operator set was deliberately limited and identical for both languages, to give a common reference point, and it is weaker than a production mutation framework; absolute scores are therefore not directly comparable to results obtained with PIT or `mutmut`, and only within-study comparisons are meaningful. The per-function mutant cap differs between the two language pipelines (20 for Java, 30 for Python), an inconsistency inherited from separate implementations. It binds two Java functions — CJ-004 and CJ-026, both sitting exactly at the 20-mutant ceiling under every Java condition — and no Python function, the largest there being CP-025 at 24 candidate sites against a cap of 30. Since T4 asks only whether at least one mutant is killed, the cap bounds the resolution of the continuous score for those two functions rather than their tier outcome.

6.2 Internal Validity

Sampling. Our own miner incorrectly attributed one Java target's visibility: `LocalTime.getField` is declared `protected`, but the parser matched it against a same-name declaration in a nested class and recorded it as public, so it passed the public-only filter. An audit against the real declaration line confirmed that this was the only occurrence among the 60 Java functions; we exclude it from the analysis. We report the case here because it is precisely the class-attribution error this study is about, reproduced in our own tooling: knowing about a failure mode does not protect one from it.

Concurrency. Mutation analysis overwrites source files, so running it on a shared source tree corrupts any concurrent measurement reading from that tree, silently. We mutate an isolated copy placed ahead of the original on the import path, and verify that the original tree was not modified after each run.

6.3 External Validity

Source exhaustion in Python. After applying all testability criteria, only 64 Python functions in the two projects qualify, 34 of which already appeared in our earlier dataset. A strictly held-out Python replication is therefore limited to about 30 functions unless additional repositories are added. The Python corpus is also drawn from two web-oriented libraries, which may limit external validity.

Complexity distribution. Although the band $5 \leq CC \leq 10$ is broad, most of the sampled functions fall towards its lower end (56 of 120 have $CC = 5$, more than the 40 with CC between 7 and 10 combined). Conclusions about the upper half of the band therefore rest on fewer observations.

Model and configuration. A single model at a single temperature was used; we make no claims about generalisation to other models, or even to other configurations of the same one, and the deterministic setting means we do not estimate variance across runs.

6.4 Conclusion Validity

Baseline configuration. Three factors constrain the baseline comparison. First, EvoSuite was run with a classpath containing only the module's own compiled classes; with the full dependency classpath its bytecode reader aborts in a way that produces no output while still reporting success, and we were unable to isolate the offending artifact. EvoSuite may therefore perform below its usual level in this configuration. Second, seven Java targets in one multi-module project could not be recompiled to Java 11 bytecode (required by EvoSuite) and fall outside its coverage. Finally, our Randoop version differs from the one used in our earlier study, so its results are compared only within this study.

Pre-registration and two deviations. The research questions, hypotheses, tool versions, budgets, and the two-branch Pynguin design were committed to the pre-registration before any baseline was run.

The first deviation added a condition. On inspection, the enriched prompt we had intended to use did not contain the worked example, making it a zero-shot prompt rather than the intended one-shot one; it therefore differed from the baseline in two respects at once — it dropped the worked example and it added the target metadata. We added a corrected condition that keeps the worked example and varies only the target specification, noted this change in the updated pre-registration with its date, and left the original hypothesis untouched. The uncorrected prompt is reported in the results only as a secondary observation.

The second deviation changed the instrument after the results had been seen, and changed it in a way that favoured our method, so we state it plainly. The Java green-test gate described above was adjusted to comply with the pre-registered definition of T2. Eight functions were affected under the baseline prompt and ten under the target-specified prompt, against two for each of the two Java baselines; overall the correction helped the model more than it helped its competitors. Three safeguards apply. The pre-correction runs were committed to the repository before the corrected measurement was run, so their timestamps preclude selecting between the two afterwards. Both settings are reported side by side for every Java comparison. And we state no conclusion that does not hold under both; where the two disagree, we report the comparison as inconclusive. The Python measurements were not re-run, as they matched the pre-registered definition of T2 from the start.

7 Conclusion

We set out to explain why a language model asked to generate tests for individual functions often fails at doing so, finding that an appreciable part of the cause lies outside the model. Of the 1,848 functions in the medium-complexity category across ten projects, only 26 % are simultaneously public and unambiguously named. That is, the rest cannot be legitimately targeted by an external test suite, and a model evaluated on its ability to find faults in these is penalized for shortcomings unrelated to its own. Recreating the sample under a fixed set of constraints, and evaluating each suite at four nested levels, drastically alters the picture.

Our main finding is a discrepancy that would be masked by a summary using one metric. Supplying the model with the target's fully qualified name, signature, and declaring class, along with a verified constructor call, raises the fraction of Python suites that reach the target from 29/60 to 42/60 ($p = 0.003$), while its effect on the fraction that finds an injected fault is not significant (16/60 to 22/60, $p = 0.33$). The intervention improves arrival, not discrimination, and would have shown as an overwhelming success if evaluated on the coverage metric alone.

The asymmetry is what allows the same change to benefit Python and not Java. The target block provides identity, and Python resolves identity at run time: import and collection failures fall from 17 to 4. Java binds identity and type at compile time, and the count of compile failures does not move at all — 38 before, 38 after, 35 of them the same functions. What remains in both languages is the need to create a receiver in a particular state that reveals the behaviour, and neither prompt supplies such a state even after the constructor call has been executed and verified.

The comparison with existing generators is not strictly favorable to one language or the other. On Python, our model vastly outperforms Pynguin under both of that tool's configurations ($p \leq 0.003$ in all comparisons). On Java the ordering reverses: both EvoSuite and Randoop detect faults on more targets than the model under either gate, though only EvoSuite reaches statistical significance, and only under the stricter gate ($p = 0.028$). A study limited to one language would have confidently concluded the opposite of what the other found.

At last, and perhaps most unexpectedly, the study's most transferable output may be a catalogue of ways a measurement can stop working without saying so. Six such defects surfaced — a hard-coded validity constant, a flag counting execution rather than success, reflection turning method names into strings the compiler never checks, two tool-version mismatches that discovered no tests while exiting successfully, and a green-test gate implemented differently in the two languages under one name. We introduced four of the six ourselves while repairing the other two, and had we trusted one of them, we would have reported a statistically significant claim in precisely the wrong direction. None produced an error message. We therefore report the sampling funnel in full, both configurations of every tool we ran in more than one, and both settings of the gate we corrected — and we advance no conclusion that does not hold under both.

Future Work

Three issues deserve further immediate attention. First, the state barrier described in Section 5.1, which neither better prompting nor a verified constructor call appears to address, may be removed by supplying an executed state: a receiver already driven into a configuration where the target's branches are reachable. Second, the Java compile failures should be classified according to the type errors that cause them, clarifying whether the typing barrier is truly distinct from the addressing barrier or merely a harder instance of it. Finally, the funnel numbers reported here come from ten projects in two languages; whether roughly three quarters of medium-complexity functions being unaddressable from outside is a property of these projects or of software more generally is not something a sample this size can settle.

References

1. Arcuri, A., Briand, L.: A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. Software Testing, Verification and

Reliability **24**(3) (2014)
2. Chen, M., Tworek, J., Jun, H., Yuan, Q., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
3. Fraser, G., Arcuri, A.: EvoSuite: Automatic test suite generation for object-oriented software. In: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE) (2011)
4. Fraser, G., Arcuri, A.: A large-scale evaluation of automated unit test generation using EvoSuite. ACM Transactions on Software Engineering and Methodology (TOSEM) **24**(2) (2014)
5. Inozemtseva, L., Holmes, R.: Coverage is not strongly correlated with test suite effectiveness. In: Proceedings of the 36th International Conference on Software Engineering (ICSE) (2014)
6. Just, R., Jalali, D., Inozemtseva, L., Ernst, M.D., Holmes, R., Fraser, G.: Are mutants a valid substitute for real faults in software testing? In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE) (2014)
7. Kerr, N.L.: HARKing: Hypothesizing after the results are known. Personality and Social Psychology Review **2**(3) (1998)
8. Lukasczyk, S., Fraser, G.: Pynguin: Automated unit test generation for Python. In: Proceedings of the 44th International Conference on Software Engineering: Companion Proceedings (ICSE Companion) (2022)
9. Lukasczyk, S., Kroiß, F., Fraser, G.: An empirical study of automated unit test generation for Python. Empirical Software Engineering **28**(2) (2023)
10. McCabe, T.J.: A complexity measure. IEEE Transactions on Software Engineering **SE-2**(4) (1976)
11. Pacheco, C., Lahiri, S.K., Ernst, M.D., Ball, T.: Feedback-directed random test generation. In: Proceedings of the 29th International Conference on Software Engineering (ICSE) (2007)
12. Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., Harman, M.: Mutation testing advances: An analysis and survey. Advances in Computers **112** (2019)
13. Schäfer, M., Nadi, S., Eghbali, A., Tip, F.: An empirical evaluation of using large language models for automated unit test generation. IEEE Transactions on Software Engineering **50**(1) (2024)
14. Shamshiri, S., Just, R., Rojas, J.M., Fraser, G., McMinn, P., Arcuri, A.: Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges. In: Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE) (2015)
15. Siddiq, M.L., Santos, J.C.S., Tanvir, R.H., Ulfat, N., Al Rifat, F., Carvalho Lopes, V.: Using large language models to generate JUnit tests: An empirical study. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE) (2024)
16. Tufano, M., Drain, D., Svyatkovskiy, A., Deng, S.K., Sundaresan, N.: Unit test case generation with transformers and focal context. arXiv preprint arXiv:2009.05617 (2020)
17. Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q.: Software testing with large language models: Survey, landscape, and vision. IEEE Transactions on Software Engineering **50**(4) (2024)
18. Yuan, Z., Lou, Y., Liu, M., Ding, S., Wang, K., Chen, Y., Peng, X.: No more manual tests? evaluating and improving ChatGPT for unit test generation. arXiv preprint arXiv:2305.04207 (2023)